

2.5.1 Linear Data Structures (Array, Linked List, Stack, Queue)

ایرے ایک جیسے عناصر کو میموری کے لگاتار بلاک میں رکھتا ہے۔

CS XII

Chapter 2: Computational Thinking & Algorithms

Lecture 12 of 16

★ Golden

2.5.1 Linear Data Structures (Array, Linked List, Stack, Queue)

★ Golden

LEARN IT

An **array** holds a collection of **similar items** (the same data type) placed next to each other in **contiguous memory** (one continuous block).

Each item has a position number called an **index**, usually starting from **0**. Because the address of any cell can be calculated from the index, you can jump straight to it. That is **direct access** (also called random access), and it is $O(1)$.

Array name: A

23	76	12	90	34	68
0	1	2	3	4	5

A[3] = 90 (index starts at 0)

Six integers in contiguous cells. Direct access: A[3] jumps to 90 without walking 23, 76 and 12.

WORKED EXAMPLE

Worked example: read by index.

```
A = [23, 76, 12, 90, 34, 68]
print(A[3])
print(A[0])
```

OUTPUT

```
90
23
```

REMEMBER IT

Index 0 is the first shelf on the row. The third item that humans call "number 4" is index 3. Formula in your head: **position from the left, minus one**.

COMMON MISTAKE

Limitation. Arrays are fast to read by index but **inflexible** to grow in the middle. Inserting a new value between 12 and 90 means shifting 90, 34 and 68 one cell to the right. That shift is $O(n)$, so arrays are a poor fit when you insert and delete in the middle all the time.

اردو میں سمجھیں

ایرے ایک جیسے عناصر کو میموری کے لگاتار بلاک میں رکھتا ہے۔ ہر عنصر کا انڈیکس (عموماً 0 سے) ہوتا ہے جس سے براہ راست رسائی $O(1)$ میں مل جاتی ہے، جیسے $A[3] = 90$ ۔ درمیان میں نیا عنصر ڈالنے کے لیے دائیں طرف شفٹ کرنا پڑتا ہے جو سست ہے۔

CHECK YOURSELF

1. What does "contiguous memory" mean?
2. In the diagram, what is $A[2]$?
3. Why is inserting in the middle of an array inefficient?

Answers: 1. The items sit in one continuous block, next to each other. 2. 12. 3. Later elements must all be shifted right to make room.

LEARN IT

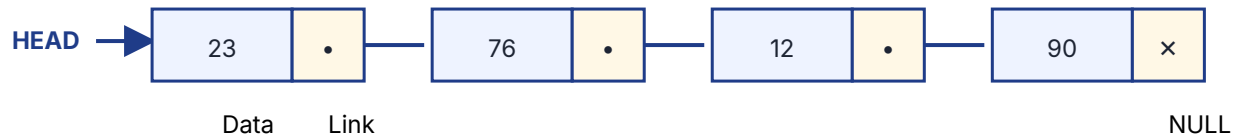
A **linked list** stores data as a chain. Unlike an array, the items are **not** one continuous block. Each separate piece is a **node** sitting somewhere in memory.

Each node has two parts:

1. the actual **data**;
2. a **link** (pointer) holding the memory address of the **next** node.

The first node is pointed to by **HEAD**. The last node's link is **NULL** (nowhere). To find an item you start at HEAD and follow links one by one — **sequential access**, $O(n)$.

Real-life example: browser history. Clicking Back follows the pointer to the previous page-node.



Follow HEAD → 23 → 76 → 12 → 90 → NULL

Four nodes. The last link is NULL, which ends the chain.

WORKED EXAMPLE

Worked example: walk from HEAD to print every data field.

```

nodes = [23, 76, 12, 90]
for data in nodes:
    print(data, end=" → ")
print("NULL")
  
```

OUTPUT

23 → 76 → 12 → 90 → NULL

There is no `nodes[2]` jump in a real linked list. The loop above models following each link once. Inserting a new node between 76 and 12 only means changing two pointers — you do not shift the rest of a big block, which is why lists beat arrays for frequent insert/delete.

REMEMBER IT

Array = row of lockers with numbers on the doors. Linked list = treasure hunt: each clue (link) tells you where the next box is.

COMMON MISTAKE

Mistake: writing `list[3]` and calling it linked-list access.

Fix: a true linked list has no index. Python's `list` type is an array-like structure. In this chapter, "linked list" means HEAD-and-pointers.

اردو میں سمجھیں

لنکڈ لسٹ زنجیر کی طرح نوڈز پر مشتمل ہے جو میموری میں بکھرے ہو سکتے ہیں۔ ہر نوڈ میں ڈیٹا اور اگلے نوڈ کا پتہ (Link) ہوتا ہے۔ پہلا نوڈ HEAD سے ملتا ہے، آخری کا لنک NULL ہوتا ہے۔ براؤزر کا Back بٹن پچھلے نوڈ کے پوائنٹر پر چلتا ہے۔ رسائی ترتیب وار ہے، انڈیکس سے چھلانگ نہیں۔

CHECK YOURSELF

1. Name the two parts of a node.
2. What does the last node's link store?
3. Why is browser history a fair analogy?

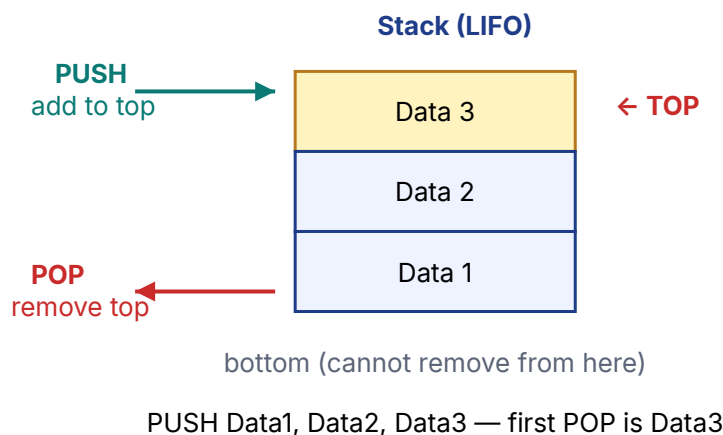
Answers: 1. Data, and a link/pointer to the next node. 2. NULL (no next node). 3. Back follows the pointer to the previous page-node instead of jumping by index.

LEARN IT

A **stack** is a linear data structure that works on the **LIFO** principle: **Last-In, First-Out**. The last element added is the first one removed. You cannot pull an item from the bottom or the middle.

- **PUSH** — add an item to the **top**.
- **POP** — remove the item from the **top**.

A **TOP** pointer always names the current top item. **Real-life example:** Undo in a text editor. Every action is piled on the stack; Undo POPs the most recent action first.



Three PUSH operations. The TOP is Data 3, so the first POP returns Data 3.

WORKED EXAMPLE

Worked example: PUSH / POP trace. Start with an empty stack. Then PUSH Data1, PUSH Data2, PUSH Data3, POP, POP.

Step	Operation	Stack (bottom → top)	TOP	Popped value
0	(empty)	[]	none	
1	PUSH Data1	[Data1]	Data1	
2	PUSH Data2	[Data1, Data2]	Data2	
3	PUSH Data3	[Data1, Data2, Data3]	Data3	
4	POP	[Data1, Data2]	Data2	Data3
5	POP	[Data1]	Data1	Data2

```
stack = []
stack.append("Data1")
stack.append("Data2")
stack.append("Data3")
print("after 3 PUSHes:", stack)
print("POP:", stack.pop())
print("POP:", stack.pop())
print("left:", stack)
```

OUTPUT

```
after 3 PUSHes: ['Data1', 'Data2', 'Data3']
POP: Data3
POP: Data2
left: ['Data1']
```

Python's `list.append` is PUSH and `list.pop()` (no index) is POP from the top.

REMEMBER IT

LIFO = Last In First Out = "last plate washed is first plate used" on a canteen pile. Undo, Redo, Back button and function calls are all stacks.

COMMON MISTAKE

Mistake: POPing from the bottom, or mixing LIFO with FIFO.

Fix: stack = plates (top only). Queue = ticket line (front leaves, rear joins). If the first POP is Data1, you drew a queue, not a stack.

سٹیک LIFO پر چلتا ہے: جو سب سے بعد میں آیا وہ سب سے پہلے نکلے گا۔ اوپر رکھنا PUSH، اوپر سے ہٹانا POP، اور TOP پوائنٹر موجودہ سب سے اوپر عنصر بتاتا ہے۔ انڈو (Undo) کی مثال: ہر عمل ڈھیر پر پڑتا ہے، انڈو سب سے تازہ عمل ہٹاتا ہے۔

EXAM TIP

Define stack + LIFO, define PUSH and POP, mention TOP, give the Undo example, and draw the vertical boxes with TOP labelled. Trace questions: "Show the stack after PUSH A, PUSH B, POP, PUSH C." Work left to right; the first POP must be B. Never remove from the middle.

CHECK YOURSELF

1. Expand LIFO.
2. After PUSH Data1, Data2, Data3, what does the first POP return?
3. Why is Undo modelled as a stack?

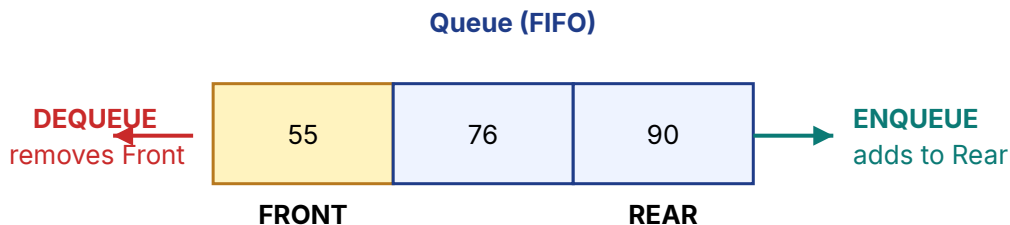
Answers: 1. Last-In, First-Out. 2. Data3. 3. Each action is PUSHed; Undo POPs the most recent action first.

LEARN IT

A **queue** organises data on the **FIFO** principle: **First-In, First-Out**. Whoever arrives first is served first. Order truly matters.

- **ENQUEUE** — add an element at the **Rear** (back).
- **DEQUEUE** — remove an element from the **Front**.

Real-life example: several documents sent to a printer. The printer prints the first document received, then the next.



Next ENQUEUE joins after 90. Next DEQUEUE removes 55.

Three jobs waiting. New jobs join at the rear; service always takes the front.

WORKED EXAMPLE

Worked example: ENQUEUE / DEQUEUE trace. Start empty. ENQUEUE 55, 76, 90, then ENQUEUE D, then DEQUEUE.

Step	Operation	Queue (Front → Rear)	Front	Rear	Removed
0	(empty)	[]	—	—	
1	ENQUEUE 55	[55]	55	55	
2	ENQUEUE 76	[55, 76]	55	76	
3	ENQUEUE 90	[55, 76, 90]	55	90	
4	ENQUEUE D	[55, 76, 90, D]	55	D	
5	DEQUEUE	[76, 90, D]	76	D	55

```
from collections import deque
q = deque()
q.append(55)
q.append(76)
q.append(90)
q.append("D")
print("after ENQUEUE D:", list(q))
print("DEQUEUE:", q.popleft())
print("left:", list(q))
```

OUTPUT

```
after ENQUEUE D: [55, 76, 90, 'D']
DEQUEUE: 55
left: [76, 90, 'D']
```

`append` is ENQUEUE at the rear; `popleft` is DEQUEUE from the front. The first DEQUEUE is 55 (the oldest), never D.

REMEMBER IT

FIFO = First In First Out = a canteen line. Front = the person being served. Rear = the person who just joined.

COMMON MISTAKE

Mistake: DEQUEUEing from the rear, or ENQUEUEing at the front.

Fix: new items only at Rear; removal only at Front. If your first removal is D, you treated the queue as a stack.

کیو FIFO پر چلتی ہے: جو پہلے آیا وہ پہلے نکلے۔ پیچھے (Rear) شامل کرنا ENQUEUE، آگے (Front) سے ہٹانا DEQUEUE ہے۔ پرنٹر کی قطار: جو فائل پہلے بھیجی گئی وہ پہلے چھپے گی۔ پہلی DEQUEUE ہمیشہ سب سے پرانا عنصر نکالتی ہے۔

EXAM TIP

Define queue + FIFO, define ENQUEUE (rear) and DEQUEUE (front), give the printer example, draw the labelled Front/Rear diagram, and trace a short sequence. Examiners mix stack and queue on the same paper: if they say "first come, first served" write Queue; if they say "Undo" write Stack.

CHECK YOURSELF

1. Expand FIFO. Where is an item added, and where is it removed?
2. After ENQUEUE 55, 76, 90, what does DEQUEUE return?
3. Why is a printer a queue, not a stack?

Answers: 1. First-In, First-Out; added at Rear, removed from Front. 2. 55. 3. Documents must print in the order they arrived (first received, first printed), not last-received first.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. An array stores items:

- a. In contiguous indexed slots of one type
- b. With arbitrary node links only
- c. As a hierarchy always
- d. As a pie

MCQ 2. A linked list stores items:

- a. In nodes with pointers to the next
- b. In one block of equal indexes only
- c. As a tree of bosses
- d. As a hash of pies

MCQ 3. A stack is:

- a. LIFO
- b. FIFO
- c. A graph
- d. A GUI

MCQ 4. A queue is:

- a. FIFO
- b. LIFO
- c. A binary tree
- d. A K-map

MCQ 5. Undo in an editor uses a:

- a. Stack
- b. Queue
- c. Pie
- d. OSI layer

MCQ 6. A printer line is a:

- a. Queue
- b. Stack
- c. AVL tree always
- d. NAND

Extra short questions

1. Define array, linked list, stack, queue in one line each.
2. One op each: push/pop, enqueue/dequeue.
3. Array vs list for insert at front.
4. Peek means:

Extra long question

1. Draw stack and queue after the sequence: insert A,B,C then remove one. Show remaining order and a real example of each. ★ Golden for stack/queue.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Array: indexed block. List: nodes+links. Stack: LIFO. Queue: FIFO.

2. Stack push/pop top. Queue enqueue rear, dequeue front.
3. Array shifts $O(n)$; list pointer change $O(1)$ if you have the head.
4. Read the top/front without removing.

Long

1. Stack remaining B,A (C popped). Queue remaining B,C (A served). Examples: back button vs ticket counter. Diagrams with top/front labelled.

Next lecture: 2.5.2 Non-Linear Data Structures (Tree, Graph)